

dbt Project Checklist

This file is a configuration reference for new dbt projects on the Bloomwell Snowflake stack. It captures every non-obvious pattern wired up in `hs-tickets-v2` — infrastructure routing, data quality, Bloomwell naming conventions, and testing architecture — so you don't have to rediscover it from scratch.

Summary Table

#	Item	What it does
1	<code>generate_database_name</code> macro	Routes every model to the correct Snowflake database (<code>SILVER</code> vs <code>SILVER_DEV</code>) based on <code>target.name</code>
2	<code>target_db()</code> helper macro	Central env-aware resolver called by <code>generate_database_name</code>
3	<code>generate_schema_name</code> macro	Uses custom schema name verbatim — no <code>target.schema</code> prefix appended
4	<code>dbt_project.yml</code> — layer routing	<code>+database</code> / <code>+schema</code> per folder so every layer lands in the right Snowflake DB
5	<code>persist_docs</code>	Pushes model + column descriptions into Snowflake object comments (Snowsight, Cortex)
6	<code>store_failures</code> + failures schema	Persists test failure rows to a dedicated <code>REFERENCE</code> schema for auditability
7	Test severity (<code>+severity: warn</code>)	Prevents test failures from blocking downstream model runs in production
8	PK/FK <code>post_hook</code> constraints	Adds Snowflake metadata constraints after full-refresh for ER diagrams and Cortex lineage
9	<code>on-run-end</code> audit macro	Appends a pipeline health snapshot to five audit tables in <code>REFERENCE</code> after every run/test/build
10	<code>vars</code> block — thresholds & sentinels	Centralises SLA hours, sentinel FK keys, and test threshold dates as dbt vars
11	Seeds in <code>REFERENCE.config</code>	Config tables loaded by <code>dbt seed</code> , consumed at runtime by the ingestion Lambda
12	Source freshness checks	<code>freshness:</code> blocks alert on stale ingestion via <code>dbt source freshness</code>
13	dbt native unit tests	YAML mock-input / expected-output tests isolated in a dedicated <code>unit_tests</code> schema
14	<code>packages.yml</code> — <code>dbt_utils</code> + <code>dbt_expectations</code>	Standard test packages; must be pinned and committed

#	Item	What it does
15	<code>transient: false</code>	Ensures Snowflake uses permanent tables with full Time Travel on Bronze/Silver/Gold
16	Bloomwell naming charter	snake_case, layer-in-schema-not-name, <code>dim_/fct_/mrt_</code> prefixes, <code>_key</code> vs <code>_id</code> columns
17	Silver/Gold grain + column docs	Every Silver/Gold model requires a grain statement and all-column descriptions (CI gate)
18	Generic custom tests	Reusable test macros in <code>tests/generic/</code> (<code>not_in_future</code> , <code>non_negative_milliseconds</code> , etc.)
19	Singular assertion tests	Cross-model sanity checks in <code>tests/*.sql</code> that don't fit a single model's schema YAML
20	<code>profiles.yml</code> — dev/prod targets	<code>target: dev</code> default; prod has higher thread count; database is the fallback only
21	<code>+tags</code> per layer	Enables <code>--select tag:prod / --exclude tag:dev</code> to skip unit tests in production jobs
22	Sentinel row pattern	Every <code>dim_</code> table contains a hardcoded <code>-1 / 'n.a.'</code> row so FK joins never produce NULLs
23	Ghost row pattern	Ephemeral <code>int_dim_*</code> models detect source IDs referenced in facts but absent from the dim source, and inject placeholder rows before the dim is built
24	<code>coalesce_fk_str</code> macro	Normalises raw VARCHAR FK columns (NULL, '', 'None') to the sentinel string <code>'-1'</code> in bronze cleaning models
25	<code>relationships</code> test with <code>where</code> filter	Validates FK integrity while excluding the sentinel value so <code>-1</code> rows never trigger false failures

Per-Item Configuration

1 & 2 & 3 — Env-Routing Macros

These three macros work together. Without them every model lands in the profile's default database with a prefixed schema name — both wrong.

`macros/target_db.sql` — the single source of truth for env detection:

```
{% macro target_db(base_name) -%}
  {% set env = var('env', 'prod' if (target.name | lower == 'prod') else 'dev') -%}
  {% if env | lower == 'prod' -%}
    {{ base_name | upper }}
  {% else -%}
```

```

    {{ (base_name ~ '_DEV') | upper }}
  {% endif -%}
{% endmacro %}

```

macros/generate_database_name.sql — overrides dbt's default database routing:

```

{% macro generate_database_name(custom_database_name, node) -%}
  {% if custom_database_name is none -%}
    {{ target_db('STAGING') }}
  {% else -%}
    {{ target_db(custom_database_name | trim) }}
  {% endif -%}
{% endmacro %}

```

macros/generate_schema_name.sql — prevents dbt from prepending **target.schema** to custom schema names:

```

{% macro generate_schema_name(custom_schema_name, node) -%}
  {% if custom_schema_name is none -%}
    {{ target.schema }}
  {% else -%}
    {{ custom_schema_name | trim }}
  {% endif -%}
{% endmacro %}

```

Result: A model configured with **+database: SILVER** lands in **SILVER** on prod and **SILVER_DEV** on dev. No manual renaming needed.

4 — **dbt_project.yml** Layer Routing

Wire each medallion layer to its own database and schema in the **models:** block. Also set default materialisation and disable transient tables here.

```

# dbt_project.yml
models:
  <project_name>:
    +persist_docs:
      relation: true
      columns: true
    1_bronze:
      +database: BRONZE
      +schema: <SOURCE_SYSTEM>    # e.g. HUBSPOT__TICKET
      +materialized: table
      +transient: false
      +tags: ['hourly', 'bronze', 'prod']
    2_silver:

```

```

+database: SILVER
+schema: <SOURCE_SYSTEM>
+materialized: table
+transient: false
+tags: ['hourly', 'silver', 'prod']
3_gold:
+database: GOLD
+schema: <SOURCE_SYSTEM>
+materialized: table
+transient: false
+tags: ['hourly', 'gold', 'prod']
unit_tests:
+database: STAGING
+schema: UNIT_TESTS
+tags: ['dev']

```

Note: `generate_database_name` macro (item 1) automatically appends `_DEV` in dev targets — you only write the prod name here.

5 — `persist_docs`

Pushes every model description and column description into Snowflake's object comments. Required for Snowsight documentation and Cortex AI metadata.

```

# dbt_project.yml – under models: <project_name>:
+persist_docs:
  relation: true
  columns: true

```

No additional configuration needed. Descriptions in schema YAML files are automatically synced on every `dbt run`.

6 — `store_failures` + Failures Schema

Stores failing test rows as tables so you can query them post-run. Route all failures to a single schema in `REFERENCE` for the audit macro (item 9) to read.

```

# dbt_project.yml
tests:
+severity: warn
+store_failures: true
+database: REFERENCE
+schema: <PROJECT_NAME>__DBT_TEST_FAILURES

```

Result: A failing `unique` test on `fct_ticket_status_history.ticket_status_key` creates a table `REFERENCE`.

<PROJECT_NAME>__DBT_TEST_FAILURES.unique_fct_ticket_status_history_ticket_status_key containing the duplicate rows.

7 — Test Severity

Setting `+severity: warn` globally means test failures are logged but do not abort the run. Override to `error` per column for hard blockers (e.g. PK uniqueness on surrogate keys).

```
# dbt_project.yml – global default
tests:
  +severity: warn

# models/2_silver/config.yml – override to error for critical keys
models:
  - name: dim_pipeline_stage
    columns:
      - name: pipeline_stage_key
        data_tests:
          - not_null:
              config:
                severity: error
          - unique:
              config:
                severity: error
```

8 — PK/FK `post_hook` Constraints

Snowflake does not enforce PK/FK constraints — they are metadata for ER diagram tools and Cortex AI. Add them via `post_hook` after every full-refresh.

Option A — In the SQL model file (current `hs-tickets-v2` pattern for most models)

```
-- models/2_silver/fct_ticket_status_history.sql
{{
  config(
    materialized='incremental',
    unique_key='ticket_status_key',
    incremental_strategy='merge',
    post_hook=[
      "{% if not is_incremental() %}
        ALTER TABLE {{ this }} ADD CONSTRAINT pk_fct_ticket_status_history
          PRIMARY KEY (ticket_status_key);
        ALTER TABLE {{ this }} ADD CONSTRAINT
fk_fct_ticket_status_history_pipeline_stage
          FOREIGN KEY (pipeline_stage_key)
```

```

REFERENCES {{ ref('dim_pipeline_stage') }}
(pipeline_stage_key);
    {% endif %}"
]
)
}}
```

Option B — In `config.yml` (preferred Bloomwell standard — keeps SQL clean)

```

# models/2_silver/config.yml
models:
  - name: dim_team
    config:
      materialized: table
      post_hook:
        - "ALTER TABLE {{ this }} ADD CONSTRAINT pk_dim_team PRIMARY KEY
(team_key)"
        - "ALTER TABLE {{ this }} ADD CONSTRAINT fk_dim_team_owner
FOREIGN KEY (owner_key) REFERENCES {{ ref('dim_owner') }}"
(owner_key)"
```

Rules:

- Wrap incremental model hooks in `{% if not is_incremental() %}` — the constraint already exists after the first run; re-adding it fails.
- Tables (non-incremental) don't need the guard.
- Add a corresponding dbt `relationships` test in schema YAML to validate FK integrity at query time.

9 — `on-run-end` Audit Macro

Runs a macro once after every `dbt run`, `dbt test`, or `dbt build` to append a pipeline health snapshot to five append-only audit tables in `REFERENCE`.

```

# dbt_project.yml
on-run-end:
  - "{{ audit_refresh_history() }}"
```

`macros/audit_refresh_history.sql` must exist. Key guard inside the macro:

```

{% if target.name | lower != 'prod' %}
    {{ log("audit_refresh_history: skipped (non-prod target)", info=True) }}
    {{ return('') }}
{% endif %}
```

The macro writes to five tables (all append-only — full history across runs):

Table	Content
INGESTION_HEALTH_HISTORY	Daily run count vs expected 24; WARNING if mismatch
TABLE_ROW_COUNT_HISTORY	Row counts for key source/bronze tables
DBT_TEST_SUMMARY_HISTORY	Aggregate pass/fail counts + pass-rate %
DBT_TEST_BY_TYPE_HISTORY	Breakdown by test category (not_null, unique, custom, ...)
DBT_TEST_DETAIL_HISTORY	Per-test PASS/FAIL status with failure row counts

Configure the destination database and schema via dbt vars:

```
# dbt_project.yml
vars:
  audit_database: REFERENCE
  audit_schema: <PROJECT_NAME>__AUDIT
```

10 — vars Block

Centralise shared constants so they are overridable per environment and testable without code changes.

```
# dbt_project.yml
vars:
  # Test threshold – used in custom date comparison tests
  threshold_date: '2024-01-01'

  # SLA business hours (inclusive start, exclusive end)
  sla_start_hour: 8
  sla_end_hour: 18

  # Sentinel FK value for unknown/deleted dimension rows
  # Generate with: SELECT MD5('-1' || '|' || '-1')
  sentinel_pipeline_stage_key: "fb8cb8ac051a4297cb62c7517f1564f4"

  # Audit destination (used by audit_refresh_history macro)
  audit_database: REFERENCE
  audit_schema: <PROJECT_NAME>__AUDIT
```

Reference in models: `{{ var('sla_start_hour') }}`. Override at runtime: `dbt run --vars '{"sla_start_hour": 9}'`.

11 — Seeds in REFERENCE.config

Seeds are CSV files checked into git and loaded as Snowflake tables. Use them for static config tables consumed by the ingestion Lambda at runtime (pipeline config, property allow-lists, etc.).

```
# dbt_project.yml
seeds:
  +quote_columns: false
  <project_name>:
    +database: REFERENCE
    +schema: config
    +tags: ['seeds', 'prod']
```

Important: The `generate_schema_name` macro (item 3) must be in place, otherwise dbt will prepend `target.schema` to `config` and the seed lands in the wrong schema.

Run with: `dbt seed` (or `dbt seed --select pipeline_config`).

12 — Source Freshness Checks

Declare `freshness` on source tables to enable `dbt source freshness`. This detects stale ingestion before downstream models run on outdated data.

```
# models/0_staging/config.yml
version: 2

sources:
  - name: hubspot_raw
    database: bloomwell_staging
    schema: hubspot__ticket
    tables:
      - name: stg_tickets
        description: "Raw HubSpot tickets. Grain: one row per API load per ticket."
        config:
          freshness:
            warn_after: {count: 2, period: hour}
            error_after: {count: 5, period: hour}
            loaded_at_field: "TO_TIMESTAMP_LTZ(loaded_at)"
```

Run with: `dbt source freshness`. Integrate into CI as a pre-run step to block stale runs.

13 — dbt Native Unit Tests

Unit tests mock upstream model inputs and validate expected outputs without touching production data. Isolate them in a dedicated schema so they never run in production jobs.

dbt_project.yml routing:


```
models:
  <project_name>:
    unit_tests:
      +database: STAGING          # → STAGING_DEV in dev via target_db()
      +schema: UNIT_TESTS
      +tags: ['dev']
```

YAML structure (models/unit_tests/my_model.yml):

```
unit_tests:
  - name: test_my_model_sla_breach
    model: fct_accumulating_ticket_status
    given:
      - input: ref('int_ticket_renamed')
        rows:
          - {ticket_id: 1, created_at: '2024-01-01 08:00:00', pipeline_stage_key:
'abc123'}
      - input: ref('dim_pipeline_stage')
        rows:
          - {pipeline_stage_key: 'abc123', stage_name: 'Open', is_closed: false}
    expect:
      rows:
        - {ticket_id: 1, is_sla_breached: false}
```

Running locally: `dbt test --select tag:dev`

Exclude from prod jobs: `dbt build --exclude tag:dev`

14 — packages.yml

Pin both packages. Unversioned packages resolve to latest at `dbt deps` time and can introduce breaking changes.

```
# packages.yml
packages:
  - package: dbt-labs/dbt_utils
    version: "1.3.0"
  - package: calogica/dbt_expectations
    version: "0.10.4"
```

After adding or updating: `dbt deps`

Key tests from these packages used in this project:

Package	Test	Use case
dbt_utils	<code>unique_combination_of_columns</code>	Composite PK uniqueness on fact tables
dbt_utils	<code>expression_is_true</code>	Guard numeric columns ≥ 0
dbt_expectations	<code>expect_table_row_count_to_be_between</code>	Minimum row count sanity check on sources
dbt_expectations	<code>expect_column_pair_values_A_to_be_greater_than_B</code>	e.g. <code>closed_at > created_at</code>

15 — `transient: false`

Without this, Snowflake creates transient tables by default for dbt-managed objects. Transient tables have no Fail-safe storage and limited Time Travel (0 or 1 day). Set `transient: false` on every Bronze/Silver/Gold layer.

```
# dbt_project.yml – inside each layer block
+transient: false
```

Verify: `SHOW TABLES LIKE '%<model_name>%'` IN DATABASE SILVER; — the `is_transient` column should be N.

16 — Bloomwell Naming Charter

Schema organisation

Layer	Schema pattern	Example
Bronze	<code>BRONZE.<SOURCE_SYSTEM></code>	<code>BRONZE.HUBSPOT__TICKET</code>
Staging (views)	<code>STAGING.<PURPOSE></code>	<code>STAGING.HUBSPOT_PREP</code>
Silver	<code>SILVER.<SUBJECT_AREA></code>	<code>SILVER.APPOINTMENTS</code>
Gold	<code>GOLD.<USE_CASE></code>	<code>GOLD.FINANCE_REPORTING</code>
Playground	<code>TESTING.*</code> (views only)	<code>TESTING.dev_thorsten</code>

Table prefixes

Layer	Prefix	Example
Bronze	(none)	<code>BRONZE.HUBSPOT__TICKET.contacts</code>

Layer	Prefix	Example
Staging	<source>__<entity> (double underscore)	hubspot__contacts
Silver — Dimensions	dim_	dim_patient
Silver — Facts	fct_	fct_prescription
Silver — Bridge	bridge_	bridge_patient_diagnosis
Gold — Marts	mrt_	mrt_monthly_prescription_volume

Key column convention

Type	Suffix	Example
Surrogate key (PK/FK)	_key	patient_key — generated MD5 hash
Business key (from source)	_id	hubspot_contact_id — source system ID

_key and _id are **not interchangeable**. FKs always reference _key columns.

Column suffixes

Data type	Suffix	Example
Timestamp (with time)	_at	created_at
Date (no time)	_date	birth_date
Boolean	is_ / has_ prefix	is_active, has_prescription
Monetary	_amount	service_amount
Count	_count / _quantity	appointment_count
Percentage	_pct / _ratio	churn_pct
Duration	_duration_{unit}	call_duration_seconds
CET timestamp (Gold only)	_cet postfix	service_date_cet

Naming don'ts

Don't	Do instead
Repeat layer in table name (brz_hubspot_contacts)	contacts — schema already says BRONZE.HUBSPOT__TICKET
Use non-universal abbreviations (appt_key, rx_dt)	appointment_key, prescription_date
Encode SCD type in name (dim_patient_scd2)	dim_patient — document in YAML

Don't	Do instead
Add version numbers (<code>dim_patient_v2</code>)	Replace the model
Use reserved keywords as column names (<code>date</code> , <code>type</code>)	<code>birth_date</code> , <code>appointment_type</code>
Materialise staging as table	Views or ephemeral only

17 — Silver/Gold Grain + Column Docs

CI fails if Silver or Gold models have missing descriptions or undocumented columns.

Grain statement format:

```
# models/2_silver/config.yml
models:
  - name: fct_prescription
    description: >
      Grain: one prescription event per patient per doctor per prescription_date.
      One row per prescription issued. Source: BRONZE.ERP.prescriptions.
    columns:
      - name: prescription_key
        description: "Surrogate key. MD5-hash over patient_id, doctor_id,
prescription_date."
        data_tests:
          - unique:
              config:
                severity: error
          - not_null:
              config:
                severity: error

      - name: patient_key
        description: "FK to dim_patient (patient_key)."
        data_tests:
          - not_null
          - relationships:
              to: ref('dim_patient')
              field: patient_key

      - name: prescription_date
        description: "Date the prescription was issued (DATE). Never null."
        data_tests:
          - not_null
```

Rule: Every column must have a `description`. Every `_key` column must have `unique` + `not_null` tests. Every FK column must have a `relationships` test.

18 — Generic Custom Tests

Place reusable test macros in `tests/generic/`. dbt discovers them automatically — no import needed.

```
tests/
  generic/
    test_not_in_future.sql
    test_non_negative_milliseconds.sql
    test_valid_duration_format.sql
```

Pattern:

```
-- tests/generic/test_not_in_future.sql
{% test not_in_future(model, column_name) %}

select {{ column_name }}
from {{ model }}
where {{ column_name }} is not null
      and {{ column_name }} > current_timestamp()

{% endtest %}
```

Usage in schema YAML:

```
columns:
  - name: created_at
    data_tests:
      - not_in_future
```

19 — Singular Assertion Tests

Place cross-model sanity checks in `tests/*.sql`. These return rows when the assertion fails and zero rows when it passes — standard dbt test contract.

```
tests/
  assert_fct_accumulating_min_row_count.sql
  assert_gold_dims_no_nulls.sql
  assert_brz_tickets_non_negative_duration.sql
```

Example — minimum row count guard:

```
-- tests/assert_fct_accumulating_min_row_count.sql
-- Fails if fct_accumulating_ticket_status drops below 1 000 rows.
-- Indicates a catastrophic full-refresh or broken dedup step.
```

```
select count(*) as actual_row_count
from {{ ref('fct_accumulating_ticket_status') }}
having count(*) < 1000
```

Example — multi-model null check (cross-dim):

```
-- tests/assert_gold_dims_no_nulls.sql
select 'dim_owner' as dim_name, 'owner_name' as failing_column, count(*) as
null_count
from {{ ref('dim_owner') }} where owner_name is null having count(*) > 0
union all
select 'dim_team', 'team_name', count(*)
from {{ ref('dim_team') }} where team_name is null having count(*) > 0
```

20 — profiles.yml — Dev/Prod Targets

```
# dbt/profiles.yml
<project_name>:
  target: dev          # always dev by default – never accidentally run against
                        prod

  outputs:
    dev:
      type: snowflake
      account: <ACCOUNT_LOCATOR>
      user: <USER>@bloomwell.de
      role: ANALYTICS_SERVICE_ROLE
      warehouse: DBT_WH
      database: STAGING_DEV      # fallback only – generate_database_name

    overrides per model
      schema: <SOURCE_SYSTEM>
      threads: 4

    prod:
      type: snowflake
      account: <ACCOUNT_LOCATOR>
      user: <USER>@bloomwell.de
      role: ANALYTICS_SERVICE_ROLE
      warehouse: DBT_WH
      database: STAGING          # fallback only – generate_database_name

    overrides per model
      schema: <SOURCE_SYSTEM>
      threads: 8
```

Key points:

- `database` in the profile is the fallback for models with no `+database` config. With `generate_database_name` in place, it is only used for the `none` branch (models with no explicit database assignment).
- `target: dev` default prevents accidental prod runs.
- Prod threads: 8 (higher concurrency for scheduled runs). Dev: 4.
- For key-pair auth in CI: add `private_key_path: .secrets/rsa_key.p8` (never commit the key file).

21 — `+tags` Per Layer

Tags enable selective execution in CI and local development.

```
# dbt_project.yml – inside each layer block
1_bronze:
  +tags: ['hourly', 'hubspot', 'bronze', 'prod']
2_silver:
  +tags: ['hourly', 'hubspot', 'silver', 'prod']
3_gold:
  +tags: ['hourly', 'hubspot', 'gold', 'prod']
unit_tests:
  +tags: ['dev']
```

Common run patterns:

```
# Production job – runs all prod models, skips dev-only unit tests
dbt build --exclude tag:dev

# Run only bronze layer
dbt run --select tag:bronze

# Run only unit tests locally
dbt test --select tag:dev

# Run tests that are tagged hourly
dbt test --select tag:hourly
```

Rule: The `unit_tests` folder is always tagged `dev`. Production scheduled jobs must always use `--exclude tag:dev` or `--select tag:prod`.

22 — Sentinel Row Pattern

Every `dim_` table contains one hardcoded row with a business key of `-1` (numeric dims) or `md5('-1' || ' ' || '-1')` (surrogate key dims). This row acts as a safe FK target for tickets that arrive with no value assigned to a dimension FK column — so downstream joins always resolve and never produce NULLs.

Pattern inside the dim model (example: `dim_team.sql`):

```
-- models/2_silver/dim_team.sql
with real_teams as (
    select team_id, team_name, updated_at, loaded_at
    from {{ ref('brz_teams') }}
),

ghost_teams as (
    select team_id, team_name, updated_at, loaded_at
    from {{ ref('int_dim_team') }}          -- ghost rows injected here (see
item 23)
),

sentinel as (
    select
        -1 as team_id,
        'n.a.' as team_name,
        '1970-01-01'::timestamp_ltz as updated_at,
        current_timestamp()::timestamp_ltz as loaded_at
),

final as (
    select * from real_teams
    union all
    select * from ghost_teams
    union all
    select * from sentinel
)

select * from final
```

Sentinel values used across this project:

Dim	Business key sentinel	Surrogate key sentinel (MD5)
dim_team	team_id = -1	n/a — uses business key as PK
dim_owner	owner_id = -1	n/a
dim_channel_account	channel_account_id = -1	n/a
dim_pipeline_stage	pipeline_id = -1, stage_id = -1	md5('-1' ' ' '-1') = fb8cb8ac...
dim_specialist_involvement	ticket_id = -1	md5('-1')

Store the MD5 sentinel keys in `dbt_project.yml vars`: so they can be referenced without hardcoding the hash string in multiple places (see item 10).

In fact models, coalesce FK columns to the sentinel value before joining so a missing FK still resolves:


```
-- fct_accumulating_ticket_status.sql
coalesce(si.specialist_involvement_key, md5('-1')) as specialist_involvement_key
```

23 — Ghost Row Pattern

A ghost row is a placeholder dimension row for a source-system entity that is referenced in historical fact data but has since been deleted from the source API. Without ghost rows, those FK values would have no matching PK in the dim, causing either NULL results or broken **relationships** test failures.

The pattern involves two models per dim — run in this sequence:

```
brz_<source>  →  int_dim_<entity>  →  dim_<entity>
                (ghost detection)    (real + ghosts + sentinel)
brz_<tickets> →  (also reads here)
```

Step 1 — Ephemeral ghost detector (int_dim_team.sql**):**

```
-- models/2_silver/int_dim_team.sql
{{ config(materialized='ephemeral') }}
```

```
with ticket_team_ids as (
    -- collect every team_id referenced in ticket data (both FK columns)
    select distinct try_cast(hubspot_team_id as integer) as team_id
    from {{ ref('brz_tickets_staged') }}
    where try_cast(hubspot_team_id as integer) is not null

    union

    select distinct try_cast(hs_assigned_team_ids as integer) as team_id
    from {{ ref('brz_tickets_staged') }}
    where try_cast(hs_assigned_team_ids as integer) is not null
),

known_teams as (
    -- every team_id that actually exists in the source
    select team_id from {{ ref('brz_teams') }}
),

ghost_rows as (
    -- the diff: referenced but not in source → ghost row needed
    select
        t.team_id,
        'n.a.' as team_name,
        '1970-01-01'::timestamp_ltz as updated_at,
        current_timestamp()::timestamp_ltz as loaded_at
    from ticket_team_ids t
    where t.team_id not in (select team_id from known_teams)
```

```

        and t.team_id != -1    -- exclude sentinel; it is never in source and must
not become a ghost
    )

select * from ghost_rows

```

Step 2 — Dim model unions real rows + ghost rows + sentinel (`dim_team.sql`):

```

-- models/2_silver/dim_team.sql
final as (
    select * from real_teams    -- from brz_teams (HubSpot API data)
    union all
    select * from ghost_teams  -- from int_dim_team (deleted IDs still referenced
in facts)
    union all
    select * from sentinel     -- hardcoded -1 row (for tickets with no team
assigned)
)
select * from final

```

Key rules:

- The ghost detector reads directly from **bronze** (`brz_tickets_staged`), not from any silver model, to avoid a dependency cycle.
- Ghost rows use 'n.a.' for all string attributes and '1970-01-01' for timestamps — never NULL.
- The `and t.team_id != -1` guard prevents the sentinel value from being re-emitted as a ghost row.
- Ghost rows keep the original `team_id` (the deleted source ID) so FK joins in the fact still resolve correctly.
- The pattern applies identically for `dim_owner` (`int_dim_owner`) and `dim_channel_account` (`int_dim_channel_account`).

For SCD2 dims (`dim_pipeline_stage`), orphaned stage IDs are handled differently: the fact model (`int_ticket_history_extract_clean_pipeline_stages`) remaps the orphaned `stage_id` to the sentinel `pipeline_stage_key` via a `COALESCE` after a `LEFT JOIN`, rather than injecting ghost rows into the dim.

24 — `coalesce_fk_str` Macro

HubSpot raw data produces three distinct "missing" representations for FK columns that are stored as VARCHAR: SQL `NULL`, empty string `' '`, and the literal string `'None'` (Python `None` serialised to JSON). All three must be normalised to the sentinel string `'-1'` before any join.

`macros/macros_clean_values.sql`:

```

{% macro coalesce_fk_str(column_name, sentinel='-1') %}
-- Normalises NULL, '', and 'None' to the sentinel string '-1'.
-- Does NOT strip '0' - a FK value of '0' may be a valid business key.
-- For numeric FK columns (already cast to NUMBER in bronze), use plain COALESCE

```

```
instead.
COALESCE(NULLIF(NULLIF({{ column_name }}, ''), 'None'), '{{ sentinel }}')
{%~ endmacro %}
```

Usage in bronze cleaning models:

```
{{ coalesce_fk_str('pipeline_id') }}      as pipeline_id,
{{ coalesce_fk_str('hubspot_owner_id') }}  as hubspot_owner_id,
{{ coalesce_fk_str('hs_assigned_team_ids') }} as hs_assigned_team_ids
```

When to use which coalesce:

FK column type	Pattern
VARCHAR (raw from HubSpot JSON)	{{ coalesce_fk_str('col') }}
NUMBER (already cast in bronze)	COALESCE(col, -1)

25 — relationships Test with where Filter

The standard dbt **relationships** test checks that every FK value in a child model has a matching PK in the parent. Without a **where** filter, the sentinel value **-1** in fact FK columns would be tested against the dim — and would correctly resolve to the sentinel row. However, if the sentinel value is the business key (not a surrogate), and not all dims have a **-1** row yet, the test would produce false failures.

More importantly, the **where** filter is required to exclude sentinel values when the intent of the test is **"every real FK resolves"** rather than "every row has a FK match":

```
# models/2_silver/config.yml
- name: owner_id
  data_tests:
    - not_null:
        config:
          severity: error
    - relationships:
        arguments:
          to: ref('dim_owner')
          field: owner_id
        config:
          severity: warn
          where: "owner_id != -1" # exclude sentinel – it resolves by design,
not by data quality
```

Rule: Use **severity: error** on **not_null** (the FK column must always be populated — never raw NULL after coalescing), and **severity: warn** on **relationships** (data quality signal, not a hard blocker). The **where** clause excludes the sentinel value **-1** from the FK check.

For surrogate key FKs (e.g. `pipeline_stage_key`), the sentinel is a known MD5 hash that **is** present in `dim_pipeline_stage`, so the `where` filter is not needed — but `severity: error` is appropriate because every row must resolve:

```
- name: pipeline_stage_key
  data_tests:
    - not_null:
        config:
          severity: error
    - relationships:
        arguments:
          to: ref('dim_pipeline_stage')
          field: pipeline_stage_key
        config:
          severity: error      # no where filter needed – sentinel hash exists in
dim
```